Práctica 1. Introducción al OpenGL+GLUT

Descripción.

OpenGL es una librería independiente de la plataforma que actúa como interfaz con el hardware. La ventaja de esta arquitectura es que todo el desarrollo realizado en OpenGL deberá ser operativo bajo cualquier sistema operativo que lo soporte. No obstante, cada plataforma utiliza sistemas de ventanas propio, por ello, cada plataforma tiene sus propias librerías auxiliares para la gestión de openGL. En nuestro caso utilizaremos la librería GLAUX y después la GLUT. En esta práctica aprenderemos a abrir una ventana de contexto openGL y lo básico para el desarrollo de este tipo de proyecto.

Objetivo.

1. Crear un proyecto básico bajo Microsoft Visual Studio 17.
2. Abrir una ventana OpenGl.
3. Entender como están orientados los ejes.
4. Visualizar primitivas básicas.
5. Entender el color por vértices interpolado.
6. Nociones básicas de la cámara virtual.

Material.

Código asociado a la práctica ver campus virtual:

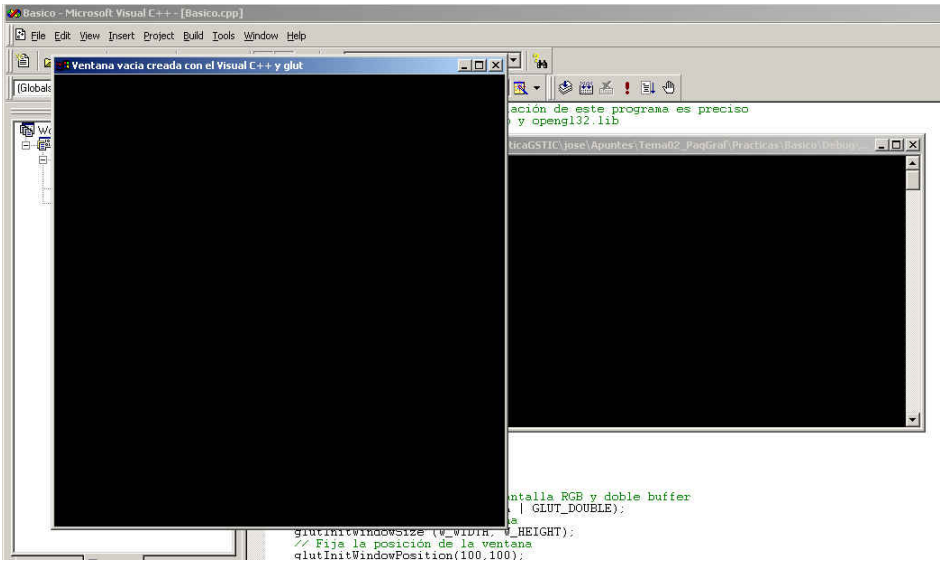
1. Librerías glut
2. Código main1a.
3. Código main1b.
4. Código main1c.
5. Codigo de funciones_ejes.c
6. Librerías glfw y glad.
7. Código main1_33a.
8. Código main1_33b (zip con varios ficheros de código).
9. Código main1_33c.
10. Fichero lectura_shaders.c.
11. Videos asociados a las prácticas.

Práctica 1.a:

Básico.

El objetivo de esta práctica abrir una ventana de contexto OpenGL con glut, Código_1.a. Recordar:

1) Nuevo proyecto.
2) Visual C++, Aplicación de consola Win32.
3) Configuración de la aplicación, proyecto vacio.
4) Añadir “archivo existente” que os paso
5) Debes incluir los *path* de las cabeceras glut.
6) Debes incluir las librerías glut32.lib y opengl32.lib, así como el *path* de donde se encuentre.



Lo que conseguimos con está aplicación es dibujar una consola en negro.

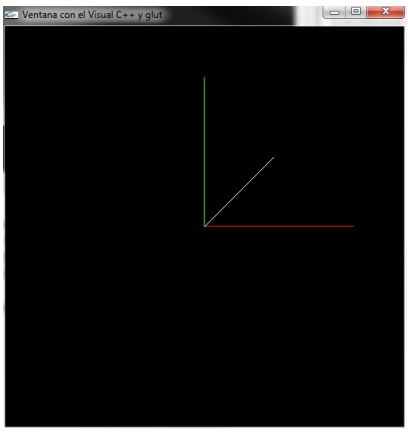
Implementa este código, analízalo, genera un ejecutable y ejecútalo. Los posibles problemas deberían ser los path´s de las librerías.

Práctica 1.b:

Generación de ejes y lazos principales.

Hacer un nuevo ejemplo en el que dibujaremos los ejes principales, así como un segmento (0, 0, 0) (1, 1, 1).

El Código ejemplo dibuja el eje X, y los principales lazos. Corresponde a dos ficheros código_main1b, y funciones ejes.c. El primero de ellos hace referencia a la función de dibujo mientras que el segundo hace referencia a los callbacks, teclado y teclas especiales y el posicionamiento de la cámara. Por ahora usa las longitudes de los ejes en proporciones similares al primero.



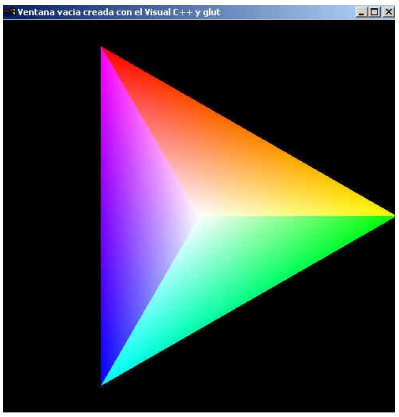
Cosas que debes hacer:

1) Coloca los tres ejes y el segmento (0, 0, 0) (1, 1, 1).
2) Prueba las teclas especiales para mover el punto de vista. No se giran los ejes sino la cámara. Aun no hemos visto esto, pero vete cogiendo la idea de los ejes y su orientación.

Práctica 1.c:

Figura coloreada.

Desarrolla un programa que realice el siguiente dibujo 2D. Se proporciona código que realiza un triángulo main1c.c.



Práctica 1_33a. Introducción al OpenGL+GLFW

Básico.

El objetivo de esta práctica es abrir una ventana de contexto openGL con glfw, Código main1_33a.c

Recordar:

1) Nuevo proyecto.
2) Visual C++, Aplicación de consola Win32.
3) Proyecto vacío.
4) Añadir “archivo existente” que os paso.
5) Incluir glac.c
6) Debes incluir los *path* de las cabeceras y librerías glfw.h, glfw.lib.

Resultado: Similar al casco de Glut.

Notas. - Implementa este código, analízalo, genera un ejecutable y ejecútalo. Los posibles problemas deberían ser los path´s de las librerías.

Práctica 1_3.3b.

Generación de ejes y lazos principales.

Hacer un nuevo ejemplo en el que dibujaremos los ejes principales, así como un segmento (0, 0, 0) (.75,.75,0.0);

El Código main1_33b dibuja el eje X, Y, Z y lee los shaders para generar los ejes mediante array de vértices, el archivo main1_33b Indices hace lo mismo solo que empleando un array con índices. Recuerda que el eje Z sale de la pantalla así que no lo veras. Por ahora usa las longitudes de los ejes en similares a los del código ejemplo. Debes incluir un segmento diagonal, (0,0,0) (.75,.75,0.0). Recuerda que debes hacerlo de dos formas. Utilizando las funciones

void **glDrawArrays**(GLenum mode, GLint first, GLsizei count);

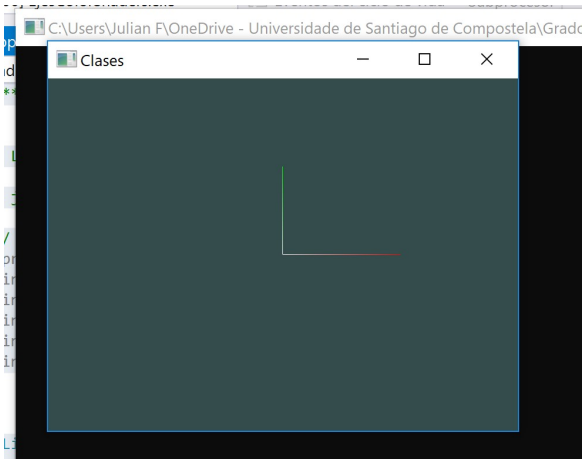
When glDrawArrays is called, it uses count sequential elements from each enabled array to construct a sequence of geometric primitives, beginning with element first. mode specifies what kind of primitives are constructed and how the array elements construct those primitives.

void **glDrawElements**(GLenum *mode*, GLsizei *count*, GLenum *type*, const GLvoid * *indices*);

When glDrawElements is called, it uses count sequential elements from an enabled array, starting at indices to construct a sequence of geometric primitives. mode specifies what kind of primitives are constructed and how the array elements construct these primitives. If more than one array is enabled, each is used. If GL_VERTEX_ARRAY is not enabled, no geometric primitives are constructed.

Resultado. Similar al del Caso de Glut.

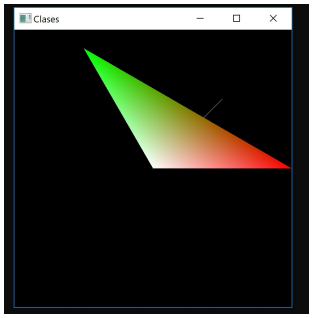
Notas. - Implementa este código, analízalo, genera un ejecutable y ejecútalo. Mira como se leer los shader y su contenido. Mira cómo se generan los, data vertex los buffers y como se configuran los atributos de los vértices. vertex data (and buffer(s)) and configure vertex atributes. Se proporciona el mismo código, pero para hacer los ejes coloreados. Añade el segmento diagonal y haz los ejes de un único color. Fíjate que los shader han cambiado.

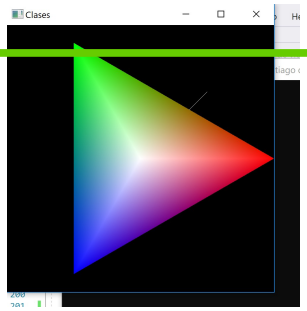


Practica 1_33.c:

Figura coloreada.

Desarrolla un programa que realice el siguiente dibujo 2D. Se proporciona el código de uno de los triángulos main1_33c.cpp.





Notas. - Implementa este código, analízalo, genera un ejecutable y ejecútalo. Mira cómo se leer los shader que deben ser iguales o similares a los anteriores, así como su contenido.

Entrega.

Esta práctica no tiene entrega.